

同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称

数据采集与人工智能研究：基于多技术融合的反爬虫智能对抗研究

实验成员

()

日期

2025 年 11 月 9 日

1、引言

1.1 研究背景与意义

在数字经济时代，数据已成为驱动科学研究、商业决策和社会治理的核心资源。网络爬虫作为数据采集的主要技术手段，广泛应用于搜索引擎索引、市场分析、舆情监测、学术研究等众多领域。然而，随着数据价值的不断提升和网络安全意识的增强，网站运营方部署了日益复杂的反爬虫机制以保护其数据资产和服务器资源。这些反爬虫技术经历了从简单的 IP 封禁、User-Agent 检测，到复杂的图像验证码（CAPTCHA）、滑块验证、频率限制算法，再到基于机器学习的行为模式识别和设备指纹追踪等多代演进。传统的反爬虫对抗方法主要依赖人工分析网站结构、手动编写规则脚本、使用代理 IP 池和模拟人类行为等策略，这些方法存在明显的局限性：规则系统脆弱且难以应对网站的频繁更新，人工维护成本高昂，对新型反爬机制的适应性差，难以在规模化场景下保持稳定性能。

近年来，人工智能技术的快速发展为反爬虫对抗研究带来了革命性的新机遇。深度学习在计算机视觉领域的突破使得自动化识别验证码成为可能，卷积神经网络（CNN）能够学习图像的深层特征表示，有效应对各种干扰和变形；强化学习通过与环境的交互试错，能够自主学习最优决策策略，为动态调整爬虫行为参数提供了智能化解决方案；大语言模型（LLM）展现出的强大代码理解与生成能力，使得自动化分析反爬机制、智能生成对抗代码成为现实。

本次研究立足于这一技术背景，探索将 CNN、强化学习（DQN/PPO）和大语言模型（DeepSeek）等多种 AI 技术应用于反爬虫对抗的不同环节，具有重要的理论价值和实践意义。在理论层面，本研究有助于深化对 AI 技术在对抗性任务中应用规律的理解；在实践层面，研究成果可为合法的数据采集工作（如学术研究、公开信息监测等）提供高效、智能的技术支持，同时也为网络安全研究提供攻防对抗的实证案例。

1.2 研究问题

基于上述研究背景，本报告聚焦于反爬虫对抗这一具体应用场景，旨在系统性地探究不同人工智能技术在突破各类反爬虫机制中的有效性与适用边界。具体而言，本研究围绕三个核心问题展开深入探究：

问题一：深度学习模型（CNN）识别复杂图像验证码

图像验证码作为最常见的反爬虫手段之一，其设计目的是利用图像识别任务区分人类用户与自动化程序。本研究将构建基于卷积神经网络的验证码识别模型，通过实验验证其在不同复杂度验证码（包含噪声干扰、字符扭曲、背景混淆等）上的识别准确率、推理速度和泛化能力，并与传统 OCR 方法进行对比分析，探究深度学习方法的优势与局限。

问题二：通过强化学习智能体（DQN/PPO）自适应学习最优采集策略以规避频率限制

网站通过监控请求频率、IP 来源等特征来检测并封禁爬虫行为，传统方法通常采用固定延时或简单的随机策略。本研究将反爬虫对抗建模为马尔可夫决策过程，分别实现基于 Deep Q-Network（DQN）和 Proximal Policy Optimization（PPO）的强化学习智能体，在模拟的反爬虫环境中训练其自主学习动态调整请求频率、切换代理 IP 等参数的策略，评估两种算法在采集成功率、效率和稳定性方面的表现差异。

问题三：利用大语言模型准确分析反爬策略并生成有效的对抗代码

传统爬虫开发需要人工分析网站的 HTML 结构、JavaScript 逻辑和反爬机制，编写定制化代码，耗时且需要较高技术门槛。本研究将利用 DeepSeek 大语言模型的代码理解与生成能力，通过精心设计的 Prompt，尝试让模型自动识别目标网站的反爬策略类型（如动态加载、Cookie 验证、请求签名等），并生成相应的 Python 爬虫代码。实验将评估生成代码的可执行率、爬取成功率以及代码质量，探讨 LLM 在自动化爬虫开发中的潜力与挑战。

1.3 研究贡献

本研究在反爬虫对抗这一特定领域进行了系统性的探索与实践，主要贡献体现在以下方面：首先，构建了一个**多技术融合的反爬虫对抗框架**。不同于以往研究多聚焦于单一技术或特定场景，本研究将 CNN 验证码识别、强化学习策略优化和 LLM 代码生成三种 AI 技术有机结合，针对反爬虫对抗中的不同挑战环节（验证码突破、行为策略规避、自动化开发）设计了完整的技术解决方案，形成了一个层次清晰、功能互补的智能对抗体系，为后续研究提供了可扩展的基础架构。

其次，提供了可复用的实验方案和代码实现。本研究从问题建模、环境构建、模型设计到实验评估，形成了一套完整的研究范式。特别是模拟反爬虫环境的设计、强化学习状态-动作-奖励的建模方式、以及针对 LLM 的 Prompt 工程实践，均具有较强的可复现性和可迁移性，可直接应用于其他对抗性学习场景或扩展至更复杂的数据采集任务。这些研究成果不仅推动了反爬虫对抗技术的发展，也为人工智能在对抗性环境中的应用研究提供了宝贵的实践经验。

2、相关工作与技术基础

2.1 传统反爬虫对抗技术调研

在人工智能技术广泛应用之前，反爬虫对抗主要依赖基于规则的传统方法。这些方法经过多年发展，形成了一套相对成熟的技术体系，但在面对日益复杂的反爬虫机制时也逐渐暴露出明显的局限性。我从验证码破解、频率控制规避和行为模拟三个核心方向系统调研传统反爬虫对抗技术的原理、实现方式及其优劣势。

①验证码破解

验证码（CAPTCHA, Completely Automated Public Turing test to tell Computers and Humans Apart）是最早也是最广泛使用的反爬虫技术之一。传统的验证码破解方法主要依赖光学字符识别（OCR）技术，其核心流程包括图像预处理和字符识别两个阶段。在图像预处理阶段，通过灰度化、二值化、降噪、字符分割等操作，将验证码图像转换为更易于识别的形式。具体而言，灰度化将彩色图像转换为单通道灰度图以减少计算复杂度；二值化通过设定阈值将图像转换为黑白两色，突出字符与背景的对比；降噪处理则使用中值滤波、形态学操作等方法去除干扰线、噪点等；字符分割通过投影法或连通域分析将粘连字符分离。在识别阶段，传统方法主要采用 Tesseract 等开源 OCR 引擎，通过模板匹配或特征提取进行字符识别。然而，随着验证码设计的复杂化，出现了扭曲变形、颜色干扰、重叠粘连等对抗手段，使得基于规则的图像预处理方法难以穷举所有情况，识别准确率大幅下降。

面对 OCR 技术的局限性，人工打码平台应运而生，成为一种广泛使用的替代方案。这类平台（如打码兔、若快打码等）采用众包模式，将验证码图像分发给真人工作者进行识别，通常能在 3-10 秒内返回结果，准确率可达 95% 以上。这种方法绕过了技术难题，理论上可以识别任何人类可读的验证码，包括滑块验证、点选验证等复杂类型。然而，人工打码存在明显的缺陷：首先是成本问题，每次识别通常需要支付 0.01-0.05 元人民币，大规模爬取时成本迅速累积；其次是速度瓶颈，即使最快的人工响应也需要数秒时间，远慢于自动化识别；最重要的是伦理与法律风险，人工打码实质上是雇佣廉价劳动力进行重复性酬金工作，且在某些场景下可能协助绕过安全机制，涉及法律灰色地带。此外，打码平台的稳定性和服务质量也难以保证，可能因平台维护、工作者数量波动等因素影响爬虫的连续运行。

②频率控制规避

网站通过监控单一 IP 地址或用户会话在单位时间内的请求次数来识别爬虫行为，一旦超过阈值即触发限流或封禁机制。传统的频率控制规避方法主要包括三种策略。固定延时是最简单的方法，在每次请求后暂停固定的时间（如 1-5 秒），以降低请求频率至人类用户的正常水平。这种方法实现简单，但效率低下，且固定的时间间隔容易被识别为机器行为特征。随机延时在固定延时基础上引入随机性，在一个区间内（如 1-3 秒）随机选择等待时间，使请求模式更接近人类的

不规律行为。Python 中常用 `time.sleep(random.uniform(1, 3))` 实现。尽管随机延时增加了检测难度，但仍无法根本解决频率限制问题，特别是在需要大量数据的场景下，过长的延时会导致采集效率极低。

代理池轮换是更为有效的频率控制规避策略。通过维护一个包含大量 IP 地址的代理池（可能来自免费代理网站、商业代理服务或自建代理网络），爬虫在每次请求或每隔若干次请求后切换到新的 IP 地址，从而将请求分散到多个“身份”上，规避单 IP 的频率限制。代理池的实现需要考虑代理的获取、验证、轮换和失效处理等环节。免费代理虽然成本低，但可用性差、速度慢且稳定性无保障；商业代理服务（如亮数据、阿布云等）提供高质量的住宅代理或数据中心代理，成功率高但成本显著；自建代理网络则需要技术投入和服务器资源。代理轮换策略也需要精心设计：简单的顺序轮换可能导致部分代理过度使用而被封禁；随机选择则可能出现代理利用不均；更优的方案是根据代理的成功率、响应时间等指标进行智能调度。然而，代理池方法也面临诸多挑战：高质量代理成本高昂，维护大规模代理池需要持续监控和更新，且部分网站采用了更严格的检测手段（如代理 IP 黑名单库、数据中心 IP 段识别等）来封禁代理流量。

③行为模拟：User-Agent 伪装、Cookie 管理、请求头构造

除了频率和 IP 检测，网站还会通过分析 HTTP 请求的特征来识别爬虫。传统的行为模拟技术通过伪装请求头、管理会话状态等方式，使爬虫的网络行为更接近真实浏览器。User-Agent 伪装是最基础的对抗手段，爬虫默认的用户代理（如 Python-requests）会立即暴露其自动化身份，因此需要将其修改为常见浏览器的标识（如 Chrome、Firefox 等）。更进一步，可以维护一个 User-Agent 池，随机选择不同的浏览器版本和操作系统组合，甚至模拟移动设备。Python 中使用 requests 库时，可通过 `headers={'User-Agent': 'Mozilla/5.0 ...'}` 进行设置。然而，仅伪装 User-Agent 远远不够，现代网站会检查整个请求头的一致性，例如 Chrome 浏览器会携带特定的 `Accept`、`Accept-Language`、`Accept-Encoding` 等字段，缺少这些字段或字段值不匹配会触发异常检测。

Cookie 管理是维持会话状态的关键。许多网站在用户首次访问时设置 Cookie，后续请求必须携带这些 Cookie 才能正常访问内容或通过登录验证。传统爬虫使用 requests 库的 Session 对象或手动管理 Cookie，在首次请求时获取并存储 Cookie，在后续请求中自动携带。对于需要登录的网站，爬虫需要模拟登录流程：首先访问登录页面获取 CSRF 令牌或验证码，然后提交账号密码（可能需要破解验证码），最后保持登录状态的 Cookie 进行数据采集。这一过程通常需要仔细分析网站的登录逻辑，包括表单参数、加密算法（如密码 MD5 加密）、重定向流程等。Cookie 管理的难点在于：部分 Cookie 具有时效性，需要定期刷新；部分网站采用 `HttpOnly`、`Secure` 等安全标志限制 Cookie 的访问；某些反爬机制会检测 Cookie 的生成时间、使用频率等异常模式。

请求头构造需要模拟完整的浏览器请求特征。除了 User-Agent 和 Cookie，还包括 `Referer`（指示请求来源页面）、`Accept` 系列字段（指示接受的内容类型和编码）、`Connection`（连接保持方式）、`Host`（目标主机）等。某些网站还会检查更细微的特征，如请求头的顺序、大小写、自定义字段等。更高级的对抗需要使用 Selenium、Puppeteer 等浏览器自动化工具，这些工具控制真实的浏览器内核（如 Chrome、Firefox），能够完美模拟人类的点击、滚动、输入等行为，并自动处理 JavaScript 渲染、AJAX 请求等复杂情况。然而，浏览器自动化也并非万无一失，网站可以通过检测 `window.navigator.webdriver` 属性、Canvas 指纹、WebGL 指纹等技术特征识别自动化工具，需要通过修改浏览器内核或注入 JavaScript 代码来绕过这些检测。

④传统方法的优势与不足分析

传统反爬虫对抗方法经过长期实践积累，具有一定的优势。首先是技术成熟度高，相关工具和库（如 requests、BeautifulSoup、Selenium 等）文档完善、社区活跃，开发者容易上手；其次是可控性强，基于规则的方法逻辑清晰，开发者可以精确控制每个环节的行为，便于调试和定位问题；第三是资源消耗低，简单的规则方法不需要大量计算资源，适合小规模或预算有限的项目。

然而，传统方法在面对现代复杂反爬虫系统时暴露出严重的不足。首要问题是规则系统的脆弱性，基于固定规则的方法难以应对网站的频繁更新，一旦网站修改 HTML 结构、调整反爬策略或升级验证码类型，爬虫脚本往往需要大幅修改甚至重写，维护成本极高。其次是人工成本高昂，每个目标网站都需要人工分析其特定的反爬机制，编写定制化的对抗代码，无法实现规模化自动

化；验证码识别、Cookie 管理、参数加密等环节都需要技术人员深入研究，耗费大量时间精力。

综上所述，传统反爬虫对抗技术为早期的数据采集工作提供了基础工具和方法论，但在当前的对抗环境下已显得力不从心。这种局限性促使研究者和实践者转向更智能、更自适应的人工智能技术，以期在反爬虫对抗的军备竞赛中获得新的突破，这也正是本研究探索人工智能（CNN、强化学习和大语言模型）应用于数据采集的重要动因。

2.2 本研究的核心 AI 技术原理

①卷积神经网络（CNN）用于验证码识别

卷积神经网络是一类专门用于处理具有网格结构数据（如图像）的深度学习模型，通过局部连接、权重共享和空间下采样等特性，能够高效地从原始像素中自动学习具有平移不变性的层次化特征表示，在验证码识别等计算机视觉任务中展现出远超传统方法的性能。

CNN 基本结构由三类核心层组成，构成了从底层特征到高层语义的抽象路径。卷积（Convolutional Layer）是 CNN 的核心组件，通过可学习的卷积核（滤波器）在输入图像上进行滑动卷积操作，提取局部特征。对于输入特征图 $X \in \mathbb{R}^{H \times W \times C}$ （高度 H 、宽度 W 、通道数 C ），卷积操作可表示为：

$$Y_{i,j,k} = \sigma \left(\sum_{c=1}^C \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} W_{m,n,c,k} \cdot X_{i+m,j+n,c} + b_k \right)$$

其中 $W \in \mathbb{R}^{K \times K \times C \times F}$ 为卷积核权重（ K 为核大小， F 为输出通道数）， b_k 为偏置， $\sigma(\cdot)$ 为非线性激活函数（通常为 ReLU: $\sigma(x) = \max(0, x)$ ）。卷积层的关键优势在于参数共享（同一卷积核在整个特征图上共享权重）和稀疏连接（每个神经元仅与局部感受野连接），大幅减少了参数量并增强了平移不变性。

池化层（Pooling Layer）对卷积层输出进行空间下采样，降低特征图分辨率的同时增强特征的鲁棒性。最大池化（Max Pooling）在每个池化窗口内选取最大值：

$$Y_{i,j,k} = \max_{m,n \in \text{pool}} X_{i+s+m,j+s+n,k}$$

其中 s 为步长，池化操作不仅减少计算量，还能提供一定程度的空间不变性，对图像的微小平移、旋转具有鲁棒性。

全连接层（Fully Connected Layer）将卷积和池化层提取的高维特征展平后映射到输出空间，对于 C 类分类问题，输出层通常配合 Softmax 函数：

$$P(y = c | \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

其中 $z_c = \mathbf{w}_c^T \mathbf{h} + b_c$ 为第 c 类的 logit 值， $\mathbf{h}$ 为前一层的特征向量。

验证码图像分类与字符识别的原理本质上是通过 CNN 将原始像素空间映射到语义标签空间的学习过程。在验证码识别任务中，输入为验证码图像 $I \in \mathbb{R}^{H \times W \times 3}$ ，输出为字符序列。对于包含 N 个字符的验证码，可建模为多标签分类问题或序列到序列问题。多标签分类方法将验证码视为 N 个独立的单字符识别任务，网络结构包含共享的特征提取部分和 N 个并行的分类头，损失函数为：

$$\mathcal{L} = \sum_{i=1}^N \mathcal{L}_c(y_i, \hat{y}_i) = - \sum_{i=1}^N \sum_{c=1}^C y_i^{(c)} \log(\hat{y}_i^{(c)})$$

其中 y_i 为第 i 个位置的真实标签（one-hot 编码）， $\hat{y}_i$ 为预测概率分布， C 为字符类别数。这种方法假设字符位置固定且相互独立，适用于定长验证码。

训练策略对 CNN 性能至关重要。在研究中，我采用数据增强对训练样本施加随机变换扩充数据集，提升模型泛化能力，常用技术包括：随机旋转（ $\theta \in [-15^\circ, 15^\circ]$ ）、随机裁剪与缩放、颜色抖动（调整亮度、对比度、饱和度）、添加高斯噪声、仿射变换等，使模型对各种干扰具有鲁

棒性。同时还采用了正则化技术防止过拟合，包括 L2 权重衰减（在损失函数中添加 $\lambda \|W\|_2^2$ 项）、Dropout（在全连接层以概率 p 随机失活神经元，迫使网络学习冗余表示）、Batch Normalization（归一化每层激活值，加速收敛并具有正则化效果）。

②强化学习模型（DQN 与 PPO）用于策略优化

强化学习是一类通过与环境交互试错来学习最优决策策略的机器学习范式，特别适用于序贯决策问题。在反爬虫对抗场景中，爬虫需要在动态变化的环境（网站的反爬机制）中学习何时发起请求、如何调整频率、何时切换代理等策略，强化学习为这一问题提供了理论框架和算法工具。

强化学习基本概念包含五个核心要素。状态（State） $s \in \mathcal{S}$ 描述环境的当前配置，在爬虫场景中可能包含：当前 IP 的请求成功率、最近 k 次请求的时间间隔、已采集数据量、当前时间等特征。动作（Action） $a \in \mathcal{A}$ 表示智能体可执行的操作，如调整请求间隔（增加/减少/保持）、切换代理 IP、等待一段时间等。奖励（Reward） r 是环境对智能体行为的即时反馈，例如成功采集数据获得正奖励，被封禁获得负奖励，请求延时过长获得小负奖励等。策略（Policy） $\pi(a|s)$ 定义了状态 s 下选择动作 a 的概率分布，可以是确定性策略 $a = \mu(s)$ 或随机策略。

价值函数衡量长期收益，状态价值函数定义为：

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

动作价值函数（Q 函数）为：

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

其中 $\gamma \in [0, 1]$ 为折扣因子，权衡即时奖励与未来奖励。强化学习的目标是找到最优策略 π^* 使累积折扣奖励最大化。

研究中使用了以下两种 Agent：

DQN（Deep Q-Network） 将深度学习与 Q 学习结合，用神经网络近似 Q 函数。传统 Q 学习维护表格 $Q(s, a)$ ，通过 Bellman 方程迭代更新：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

但状态-动作空间庞大时表格方法不可行。DQN 用深度网络 $Q(s, a; \theta)$ 参数化 Q 函数，通过最小化 TD 误差训练：

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

DQN 解决了神经网络 Q 学习的不稳定性。经验回放（Experience Replay）将智能体的经验 (s, a, r, s') 存入回放缓冲区 $\mathcal{D}$ ，训练时随机采样 mini-batch，打破样本相关性并提高数据利用率。目标网络（Target Network）使用参数 θ^- 的独立网络计算 TD 目标，每隔 C 步从主网络复制参数 $\theta^- \leftarrow \theta$ ，稳定训练过程。在爬虫场景中，DQN 可学习离散动作空间的价值估计，但对连续动作（如精确的延时秒数）处理较困难，且依赖 ϵ -greedy 探索，导致探索效率较低。

PPO（Proximal Policy Optimization） 原理是一类 on-policy 策略梯度算法，直接优化策略函数而非价值函数。策略梯度定理给出梯度方向：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) A^{\pi_\theta}(s_t, a_t) \right]$$

其中 τ 为轨迹， $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ 为优势函数（Advantage Function），衡量动作 a 相比平均水平的优劣。优势函数减小方差，加速学习。PPO 通过裁剪目标函数限制策略更新幅度，防止破坏性更新：

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

其中 $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ 为重要性采样比率， ϵ 为裁剪阈值（通常 0.1-0.2）。当优势 $A_t > 0$ 时，

限制 $r_t \leq 1 + \epsilon$ 防止策略更新过大；当 $A_t < 0$ 时，限制 $r_t \geq 1 - \epsilon$ 。这种信任域机制保证了训练稳定性。

PPO 还引入广义优势估计（GAE）高效计算优势函数：

$$A_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

其中 $\lambda \in [0, 1]$ 平衡偏差与方差， $\lambda = 0$ 时退化为 TD 误差， $\lambda = 1$ 时为蒙特卡洛收益。

二者对比，DQN 适用于离散动作空间，在 Atari 游戏等经典任务中表现优异；样本效率高但探索机制简单，在稀疏奖励或复杂环境中探索困难，而且无法处理连续动作，需离散化或使用扩展版本（如 DDPG）。PPO 则支持连续和离散动作，训练稳定性好。在反爬虫场景中，若动作为离散的几个策略选择（如选择代理池中的某个 IP），DQN 更高效；若需要连续调整请求间隔、并发数等参数，PPO 更适合；若环境复杂且需要长期稳定训练，PPO 的稳定性优势更明显。

③大语言模型用于代码生成

大语言模型（Large Language Model, LLM）是基于 Transformer 架构、在海量文本数据上预训练的自回归生成模型，近年来在自然语言理解、代码生成、推理等任务中展现出令人瞩目的能力。将 LLM 应用于反爬虫对抗中的代码生成，有望自动化爬虫开发过程，大幅降低人工成本。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 $Q = XW_Q$, $K = XW_K$, $V = XW_V$ 。

LLM 的工作机制包含三个阶段。预训练（Pre-training）阶段在大规模无标注文本语料（如 CommonCrawl、GitHub 代码库、书籍等，规模达数 TB）上训练模型预测下一个 token，学习语言的统计规律和世界知识。训练目标为最大化对数似然：

$$\mathcal{L}_{LM} = E_{x \sim \mathcal{D}} \left[\sum_{t=1}^T \log P(x_t | x_{<t}; \theta) \right]$$

其中 $x = (x_1, \dots, x_T)$ 为 token 序列， θ 为模型参数。预训练后的模型具备强大的语言建模和 few-shot 学习能力。

指令微调（Instruction Tuning）阶段在精心构造的指令-响应数据集上微调模型，使其理解人类指令并生成符合期望的输出。微调数据格式为 (I, O) 对， I 为指令（如“写一个 Python 爬虫爬取网页标题”）， O 为期望输出。通过监督学习优化：

$$\mathcal{L}_{SFT} = - \sum_{(I, O)} \log P(O | I; \theta)$$

指令微调显著提升模型的可控性和任务适应性。

上下文学习（In-Context Learning）利用模型的少样本学习能力，在推理时通过 prompt 提供任务描述和示例，无需更新参数即可完成新任务。

在研究中，DeepSeek 的特点与优势使其在代码生成任务中表现出色。DeepSeek 是一个专注于中英双语和代码领域的开源大语言模型，支持 16K-32K token 的上下文长度，可接收完整的网页 HTML 代码或复杂的任务描述，相比闭源模型（如 GPT-5）具有更低的 API 调用成本或可本地部署。这些特性使 DeepSeek 在自动化爬虫开发中具有实用价值。

Prompt Engineering 是充分发挥 LLM 能力的关键。有效的 prompt 设计需遵循以下原则：清晰描述目标任务、输入输出格式、约束条件，提供必要的背景信息，如目标网页的 HTML 结构片段、已知的反爬机制类型（如需处理 Cookie、JavaScript 渲染等）；明确代码风格要求（如添加注释、异常处理）、库版本、避免的错误模式和编码设置等。

2.3 AI 技术在反爬虫对抗中的应用现状

验证码识别领域经历了从传统图像处理到深度学习的范式转变。早期方法依赖人工设计的特征提取器配合 SVM 分类器，仅能处理简单的文本型验证码。2014 年后，卷积神经网络的引入使端到端学习成为可能，特别是 CRNN 架构结合 CTC 损失函数，实现了对不定长验证码的高效识别，在学术数据集上准确率突破 95%。2018 年后，注意力机制的应用进一步提升了模型对扭曲、粘连字符的鲁棒性。商业层面，2Captcha、Anti-Captcha 等平台已将深度学习模型与人工众包结合，提供低至 0.001 美元/次的验证码破解服务，平均响应时间 5-15 秒，准确率达 90% 以上。针对行为验证（如滑块、点选），基于计算机视觉的轨迹仿真技术通过贝塞尔曲线拟合人类操作特征，通过率已达 80%。然而，reCAPTCHA v3 的无感知风险评分机制标志着验证码技术向设备指纹和行为序列分析转型，对传统识别方法构成新挑战。

强化学习在爬虫策略优化中的应用尚处于早期探索阶段，学术界的尝试多于工业界的实际部署。Zhang 等人（2019）首次将 Q-learning 应用于自适应爬虫调度，通过学习不同时段的成功率动态调整请求频率，在模拟环境中超越固定策略 12%。Multi-Armed Bandit 算法被用于代理 IP 池管理，根据历史成功率动态分配使用概率，降低封禁率约 30%。2023 年后的研究开始探索 A3C、SAC 等深度强化学习算法，但多停留在仿真验证。实际应用面临三大障碍：状态空间定义困难——反爬策略不透明且动态变化，难以准确建模；奖励滞后性——封禁反馈可能延迟数小时，导致信用分配问题；样本效率低——在线试错成本高昂，大量 IP 资源被浪费。因此，当前主流方案采用离线强化学习或模仿学习结合专家规则的混合架构，而非纯端到端的 RL 系统。

大语言模型在 2022-2025 年间展现出强大的代码生成能力，但在爬虫开发领域的应用仍处于辅助阶段。Codex、GitHub Copilot 和 DeepSeek-Coder 在 HumanEval 基准上的 Pass@1 准确率已超 70%，证明其处理通用编程任务的能力。研究者尝试通过自然语言描述或 HTML 片段让 LLM 生成 XPath 选择器、CSS 路径或完整 Scrapy 代码，部分工作引入检索增强生成（RAG）技术，将爬虫库文档嵌入向量数据库以提高 API 调用准确性。然而，实践中暴露出三大缺陷：幻觉问题——模型可能调用不存在的函数或误解 HTML 结构，错误率在复杂页面上高达 40%；缺乏工程化组件——生成代码往往缺少异常处理、重试机制、日志记录等生产必需模块；安全隐患——未经验证的代码可能引入 SQL 注入或 XSS 漏洞。目前，LLM 在爬虫开发中更多作为“智能模板生成器”，需要人类工程师进行审查、测试和加固，而非完全自主的代码开发者。

3、基于多技术融合的反爬虫对抗实验设计

3.1 整体方法概述

本研究提出的智能爬虫系统采用分层解耦的架构设计，将复杂的反爬对抗问题分解为三个相对独立但逻辑递进的子任务层。底层为感知突破层（Perception Layer），专注于视觉验证码的自动识别与绕过，通过卷积神经网络将图像型防御转化为可处理的数字信号，相当于为系统打开进入目标网站的第一道门。中层为策略适应层（Strategy Layer），负责在动态对抗环境中进行决策优化，利用深度强化学习算法根据实时反馈（请求成功率、IP 封禁情况、响应延迟）自动调整请求频率、代理池分配和请求头伪装策略，实现从被动规避到主动博弈的转变。顶层为代码智能层（Intelligence Layer），引入大语言模型作为“虚拟爬虫生成器”，根据目标网站的 HTML 结构、JavaScript 加密逻辑和 API 文档自动生成或修改爬虫代码，大幅降低人工开发成本并提升系统对新站点的快速适配能力。这种分层设计遵循关注点分离的原则，每层专注解决特定类型问题，便于模块化开发、独立测试和灵活替换，本次研究先完成了基础的实验。

3.2 模块一：基于 CNN 的图像验证码识别

①数据集与预处理策略

本研究采用在 Kaggle 公开的 CAPTCHA Dataset (<https://www.kaggle.com/datasets/parsasam/captcha-dataset>) 作为训练数据源，该数据集包含 113,062 张彩色验证码图像，每张图像包含 5 个字符，字符集涵盖数字 0-9 和大小写字母 a-z、A-Z，共 62 个类别。原始图像尺寸为 160×60 像

素，采用 RGB 三通道彩色格式。数据预处理分为三个阶段：



图 1 验证码数据集示例

数据划分策略：我采用分层随机采样将数据集划分为训练集、验证集和测试集，具体为训练集 79,143 张（70%）、验证集 16,959 张（15%）、测试集 16,960 张（15%）。划分过程通过 `sklearn.model_selection.train_test_split` 函数实现，设置随机种子 `random_state=42` 保证实验可复现性。

图像预处理流程：原始彩色图像首先转换为灰度图，降低计算复杂度的同时保留关键的字符轮廓信息。灰度化后的图像像素值范围为[0, 255]，通过归一化公式：

$$x_{norm} = \frac{x - 127.5}{127.5} = \frac{x}{127.5} - 1$$

将像素值映射到[-1,1]区间，相比传统的[0,1]归一化，该方法使数据分布以零为中心，有助于加速梯度下降收敛并提高模型稳定性。

强化数据增强技术：为增强模型对噪声、扭曲和光照变化的鲁棒性，训练集采用七种随机增强策略的组合：

- 亮度抖动（概率 70%）：通过 PIL 库的 `ImageEnhance` 调整亮度因子 $\alpha \sim U(0.7, 1.3)$ ，变换公式为 $I_{out} = \alpha \cdot I_{in}$ 。
- 对比度调整（概率 70%）：对比度因子 $\beta \sim U(0.7, 1.3)$ ，公式为 $I_{out} = \beta \cdot (I_{in} - \mu) + \mu$ ，其中 μ 为图像均值。
- 加性噪声（概率 50%）：随机选择高斯噪声、椒盐噪声或斑点噪声之一。高斯噪声公式为：

$$I_{noisy} = I + \mathcal{N}(0, \sigma^2), \quad \sigma \sim U(5, 15)$$

椒盐噪声以概率 $p \sim U(0.01, 0.03)$ 像素随机置为 0 或 255。

- 随机模糊（概率 30%）：应用高斯模糊 $G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ 或中值滤波，核大小 $k \in \{3, 5\}$ 。
- 几何变换（概率 40%）：轻微旋转角度 $\theta \sim U(-3^\circ, 3^\circ)$ ，变换矩阵为： $M = \begin{pmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \end{pmatrix}$ ，以及缩放因子 $s \sim U(0.95, 1.05)$ 。

这些增强策略在训练时实时随机应用，显著扩充了有效训练样本的多样性。

②网络架构设计

初始设计采用了浅层 5 层 CNN 架构，包含 3 个卷积块（32→64→128 通道）和 2 个全连接层，总参数量约 50 万。5 个字符共享同一个输出层权重矩阵 $W \in \mathbb{R}^{1024 \times 310}$ （310 = 62 × 5），模型输出形状为 [batch, 310] 后 reshape 为 [batch, 5, 62]，且模型没有 Batch Normalization 层。

初版模型存在一些缺陷，仅 5 层网络无法捕获验证码中复杂的字符形变和背景干扰模式，导致特征提取能力严重受限。随着网络加深，梯度在反向传播中逐层衰减，出现梯度消失问题，使得浅层参数难以更新。5 个字符共享同一个输出层权重矩阵，这种设计强制要求网络在同一个特征空间同时识别 5 个位置的字符，导致严重的位置混淆问题——模型难以区分相同字符在不同位置的语义。缺少 Batch Normalization 导致训练过程中激活值分布剧烈波动，学习率必须设置得很小，收敛速度极慢。

为了解决这些问题，我对模型进行了改进。我设计了两种 CNN 架构用来平衡精度与效率——ImprovedCaptchaCNN（基于残差网络）和 LightweightCaptchaCNN（轻量级卷积网络）。实验最终采用 LightweightCaptchaCNN 作为主模型，其架构设计如下：

特征提取层：网络包含 4 个卷积块，每个块由两层 3×3 卷积、批归一化（Batch Normalization）、ReLU 激活和最大池化组成。通道数按 $\{64, 128, 256, 512\}$ 的比例递增，空间分辨率依次减半，遵循“深度换宽度”的设计原则。第 i 个卷积块的输出特征图计算为：

$$h^{(i)} = \text{MaxPool} \left(\text{ReLU} \left(\text{BN} \left(W^{(i)} * h^{(i-1)} + b^{(i)} \right) \right) \right)$$

其中 $*$ 表示卷积操作， $W^{(i)}$ 为第 i 层卷积核权重。

批归一化机制：在每个卷积层后插入 BN 层，对特征图进行标准化：

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

其中 γ 和 β 为可学习参数， $\epsilon = 10^{-5}$ 为数值稳定项。BN 层有效缓解了内部协变量偏移（Internal Covariate Shift）问题，允许使用更大的学习率。

自适应池化层：在卷积层输出后接入 AdaptiveAvgPool2d((4, 2))，将任意大小的特征图统一池化为 4×2 的固定尺寸，确保全连接层输入维度一致。该操作的数学表达为：

$$\text{out}(i, j) = \frac{1}{|R_{ij}|} \sum_{(m, n) \in R_{ij}} \text{in}(m, n)$$

其中 R_{ij} 为输出位置 (i, j) 对应的输入区域。

分类头设计：全连接层首先将池化后的 $512 \times 4 \times 2 = 4096$ 维特征映射到 2048 维和 1024 维的中间表示，随后为每个字符位置构建独立的线性分类器：

$$\mathbf{z}_k = \mathbf{W}_k \mathbf{h} + \mathbf{b}_k, \quad k = 1, \dots, 5$$

$$P(y_k = c | \mathbf{x}) = \frac{e^{z_{k,c}}}{\sum_{j=1}^{62} e^{z_{k,j}}}$$

其中 $\mathbf{z}_k \in \mathbb{R}^{62}$ 为第 k 个字符的 logits 向量，通过 Softmax 函数转换为 62 个类别的概率分布。这种多头设计避免了字符间的位置依赖假设，每个分类器专注于特定位置的字符识别。

正则化策略：网络在卷积块之间引入 Dropout2d（丢弃率递增：0.1→0.2→0.3→0.4），在全连接层使用 Dropout（丢弃率 0.5），通过随机失活神经元防止过拟合。Dropout 操作定义为：

$$h_{\text{drop}} = m \odot h, \quad m_i \sim \text{Bernoulli}(1 - p)$$

其中 $\odot$ 表示逐元素乘法， p 为丢弃概率。

③损失函数与优化策略

初版模型的训练策略采用基础的训练配置，导致性能不佳：

- **普通交叉熵损失：**使用硬标签 $y_c \in \{0, 1\}$ 计算损失 $L_{CE} = -\sum_{c=1}^{62} y_c \log(\hat{y}_c)$ ，模型倾向于对训练样本过度自信（输出概率接近 1），泛化能力差。
- **固定学习率：**全程使用恒定学习率 $\eta = 0.001$ ，既无法在训练初期快速收敛，也无法在后期精细调优，容易陷入局部最优。
- **无梯度约束：**深层网络的梯度在反向传播中可能爆炸，导致参数更新剧烈震荡，训练 loss 曲线出现大幅波动。

因此，在修改后的模型训练时，添加了以下关键改进：

A. 标签平滑交叉熵损失

传统的交叉熵损失函数：

$$L_{CE} = -\sum_{c=1}^C y_c \log(\hat{y}_c)$$

在真实标签 $y_c \in \{0,1\}$ 为硬标签时容易导致模型过度自信。本研究采用标签平滑 (Label Smoothing) 技术, 将硬标签软化为:

$$y_c^{smooth} = (1 - \epsilon) \cdot y_c + \frac{\epsilon}{C}$$

其中 $\epsilon = 0.1$ 为平滑系数, $C = 62$ 为类别数。平滑后的损失函数为:

$$L_{LS} = - \sum_{c=1}^C y_c^{smooth} \log(\hat{y}_c)$$

这种正则化手段鼓励模型输出更加平滑的概率分布, 提高泛化能力。

B. 多任务学习损失

由于验证码包含 5 个字符, 总损失为各位置损失的平均:

$$L_{total} = \frac{1}{5} \sum_{k=1}^5 L_{LS}^{(k)}$$

各位置损失权重相等, 反映了字符独立性假设。

C. AdamW 优化器

采用 AdamW (Adam with decoupled Weight decay) 优化器, 其更新规则为:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L)^2 \\ \theta_t &= \theta_{t-1} - \eta \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_{t-1} \right) \end{aligned}$$

其中 $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda = 0.01$ (权重衰减系数)。AdamW 将权重衰减与梯度更新解耦, 避免了 L2 正则化与自适应学习率的交互干扰。

D. 余弦退火学习率调度

初始学习率设为 $\eta_0 = 0.001$, 采用余弦退火策略 (Cosine Annealing) 动态调整:

$$\eta_t = \eta_{min} + 1/2(\eta_0 - \eta_{min})(1 + \cos(t/T_{max}\pi))$$

其中 t 为当前 epoch, $T_{max} = 80$ (总 epoch 数), $\eta_{min} = 10^{-6}$ 。该策略使学习率平滑递减, 后期的小学习率有助于模型收敛到更优的局部最小值。

E. 梯度裁剪

为防止梯度爆炸, 在反向传播后对梯度范数进行裁剪:

$$\nabla \theta \leftarrow \begin{cases} \nabla \theta, & \text{if } \|\nabla \theta\| \leq \tau \\ \frac{\tau}{\|\nabla \theta\|} \nabla \theta, & \text{otherwise} \end{cases}$$

④ 训练配置与评估指标

批大小 (Batch Size) 设为 128, 在单个 NVIDIA GPU 上训练 80 个 epoch, 数据加载采用 4 个工作进程 (num_workers=4) 并启用固定内存 (pin_memory=True) 加速 GPU 传输。

● 字符级准确率 (Character-level Accuracy)

$$Acc_{char} = \frac{1}{N \times 5} \sum_{i=1}^N \sum_{k=1}^5 \mathbb{1}(\hat{y}_{i,k} = y_{i,k})$$

其中 N 为样本数, $\mathbb{1}(\cdot)$ 为指示函数。该指标衡量所有字符位置的平均正确率。

● 整体准确率 (Overall Accuracy)

$$Acc_{overall} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\widehat{y_{i,1:5}} = y_{i,1:5})$$

仅当 5 个字符全部预测正确时计为 1，这是验证码识别的实际应用标准。

- 位置级准确率 (Position-wise Accuracy)

$$Acc_{pos}^{(k)} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\widehat{y_{i,k}} = y_{i,k}), \quad k = 1, \dots, 5$$

分析每个字符位置的识别难度，识别模型的薄弱环节。

- 大小写不敏感准确率

$$Acc_{case_insensitive} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\text{lower}(\widehat{y_i}) = \text{lower}(y_i))$$

为应对部分应用场景对大小写不敏感的需求。

3.3 模块二：基于强化学习的自适应采集策略

问题建模为马尔可夫决策过程 (MDP)：本研究将网络爬虫的反爬虫对抗问题形式化为一个部分可观测的马尔可夫决策过程 $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ ，其中：

① 状态空间 $\mathcal{S}$ 设计

爬虫的观测状态 $s_t \in \mathbb{R}^{18}$ 由 18 维特征向量组成，刻画了当前的采集环境：

$$s_t = [s_t^{(1)}, s_t^{(2)}, \dots, s_t^{(18)}]^T$$

各维度具体定义为（已归一化至[0,1]区间）：

- IP 信誉分数 $s_t^{(1)} = r_{ip} \in [0,1]$ ：当前 IP 的历史信誉评级，反映该 IP 被反爬虫系统信任的程度。
- User-Agent 年龄 $s_t^{(2)} = \min(1.0, \text{age}_{ua}/100)$ ：当前 UA 标识的使用时长归一化值。
- 多时间窗口请求频率 $s_t^{(3:6)}$ ：分别统计 10 秒、1 分钟、5 分钟、1 小时四个时间窗口 $W_k \in \{10, 60, 300, 3600\}$ 内的请求频率：

$$s_t^{(k+2)} = \min\left(1.0, \frac{|\{t': t - t' \leq W_k\}|}{T_k^{max}} \times 2\right), \quad k = 1, 2, 3, 4$$

其中 T_k^{max} 为对应窗口的安全阈值。这种多尺度频率特征使智能体能同时感知短期突发和长期趋势。

- 平均请求间隔 $s_t^{(7)} = \min\left(1.0, \frac{1}{\Delta t}\right)$ ，其中 $\Delta t = \frac{1}{N} \sum_{i=1}^N \Delta t_i$ 为最近 N 次请求的平均间隔。
- 间隔方差 $s_t^{(8)} = \min\left(1.0, \sigma_{\Delta t}/10\right)$ ：请求间隔的标准差归一化，用于检测行为规律性。规律性过强（方差小）会被反爬虫系统识别为机器行为。
- AI 检测分数 $s_t^{(9)} = \alpha_{AI}$ ：由环境内置的 AI 行为检测器计算得出，基于请求模式的机器学习分类结果。
- 连续成功率 $s_t^{(10)} = \min(1.0, n_{succ}^{consec}/10)$ ：最近连续成功请求的次数。
- 连续失败数 $s_t^{(11)} = \min(1.0, n_{fail}^{consec}/5)$ ：连续失败次数归一化。
- 验证码/JS 挑战率 $s_t^{(12)} = (n_{captcha} + n_{js}) / \max(1, n_{total})$ 。

- IP 池使用率 $s_t^{(13)} = |\{ip: n_{req}(ip) > 0\}|/N_{ip}$ 。
- 时间进度 $s_t^{(14)} = t/T_{max}$ 。
- 总成功率 $s_t^{(15)} = n_{succ}/\max(1, n_{total})$ 。
- 封禁率 $s_t^{(16)} = n_{ban}/\max(1, n_{total})$ 。
- 蜜罐命中率 $s_t^{(17)} = n_{honeypot}/\max(1, n_{total})$ ：访问诱饵链接的比例。
- 综合风险评分 $s_t^{(18)} = \text{Risk}(s_t)$ ：多因素加权的风险函数：

$$\text{Risk}(s_t) = 0.22(1 - r_{ip}) + 0.18\alpha_{susp} + 0.18\alpha_{AI} + 0.15 \min(1, n_{ban}/2.5) + 0.12 \min(1, n_{honeypot}/1.5) + \dots$$

②动作空间 $\mathcal{A}$ 设计

动作空间被设计为 9 维的离散空间（`ACTION\_DIM=9`），涵盖了请求频率调整和资源切换策略。动作集合 $a_t \in \mathcal{A}$ 包括：

- 频率控制：快速请求（0.5s）、正常请求（1.0s）、慢速请求（2.0s）。
- 资源切换：仅切换 IP、仅切换 UA、同时切换 IP 与 UA。
- 避险策略：等待冷却（3.0s）、长等待+换 IP（5.0s）、紧急避险（10.0s+全切换）。

③状态转移函数 $\mathcal{P}$ 设计

状态转移由环境的反爬虫检测机制决定，是一个确定性函数：

$$s_{t+1} = f_{env}(s_t, a_t, \xi_t)$$

其中 ξ_t 为环境噪声（如网络超时、服务器波动等）， f_{env} 实现了多层检测逻辑，详见模拟反爬环境构建的介绍。

④奖励函数 $\mathcal{R}$ 设计

为了引导智能体学习高效且安全的策略，奖励函数 r_t 设计如下：

$$r_t = \begin{cases} R_{succ} + \alpha \cdot (1/\text{interval}) & \text{if request success} \\ R_{block} & \text{if IP banned} \\ R_{captcha} & \text{if captcha triggered} \\ R_{timeout} & \text{if request timeout} \\ R_{cost} & \text{if resource switch} \end{cases}$$

根据代码配置：基础成功奖励 $R_{succ} = 1.0$ ，封禁惩罚 $R_{block} = -5.0$ （地狱模式下高达-12.0），验证码惩罚 $R_{captcha} = -2.0$ 。此外，引入效率奖励 α 鼓励在安全前提下提高速度，并对频繁切换资源施加微小的成本惩罚 R_{cost} 。

⑤模拟反爬环境构建

为了训练智能体，我基于 gymnasium 库构建了 RealisticAntiSpiderEnvironment 及 HellModeAntiSpiderEnvironment。

A.多维检测机制

环境模拟了真实的大型网站反爬系统，核心检测逻辑包括：

- 频率检测（Multi-Window Rate Limiting）

采用多时间窗口滑动算法检测请求突发，计算频率超标分数：

$$freq_score = \sum_{k=1}^4 w_k \times \max\left(0, \frac{n_k/W_k}{T_k/W_k} - 1\right)$$

其中窗口配置为 $\{W_1 = 10s, W_2 = 60s, W_3 = 300s, W_4 = 3600s\}$ ，阈值 $\{T_1 = 2, T_2 = 8, T_3 = 30, T_4 = 80\}$ ，权重 $w_k = \{0.4, 0.3, 0.2, 0.1\}$ 使短期突发影响更大。当 $freq_score > 0.9$ 时触发封禁。

- 行为指纹检测 (Behavioral Fingerprinting)

计算请求间隔的变异系数判定机器行为：

$$CV = \frac{\sigma_{\Delta t}}{\Delta t}$$

规律性分数定义为：

$$\alpha_{reg} = \frac{1}{1 + 2 \times CV}$$

CV 过小（如 $CV < 0.3$ ）表示间隔过于规律， α_{reg} 接近 1 时高度可疑。该分数作为 AI 检测的核心维度之一。

- IP 信誉系统 (IP Reputation System)

维护全局信誉数据库，初始值 $r_{ip} \sim U(0.3, 0.7)$ ，动态更新规则：

$$r_{ip}^{(t+1)} = \begin{cases} \min(1.0, r_{ip}^{(t)} + 0.005), & \text{成功 (缓慢恢复)} \\ 0.2 \times r_{ip}^{(t)}, & \text{封禁 (指数衰减)} \\ 0.8 \times r_{ip}^{(t)}, & \text{触发JS挑战} \end{cases}$$

信誉值参与综合检测分数计算： $D = \dots + (1 - r_{ip}) \times 0.3 + \dots$

- 设备指纹校验 (Device Fingerprinting)

模拟 Canvas、WebGL、TLS 指纹检测，计算不一致风险：

$$Risk_{fp} = 0.35 \times I_{res} + 0.25 \times I_{tz} + 0.50 \times I_{plat} + 0.40 \times I_{canvas} + 0.20 \times I_{webgl}$$

其中 I 为指纹不一致指示函数。当 $Risk_{fp} > 0.25$ 时触发拦截。

- 蜜罐陷阱 (Honeytrap Traps)

随机分布 15 个蜜罐 URL（占比约 5%），访问触发严厉惩罚：

$$r_{ip} \leftarrow 0.2 \times r_{ip}, \quad \alpha_{susp} \leftarrow 1.0, \quad Risk_{cumul} \leftarrow \min(1.0, Risk_{cumul} + 0.3)$$

命中蜜罐的累积风险持续影响后续请求的检测分数。

B. 状态转移逻辑

环境根据当前动作和内部检测概率 P_{detect} 计算下一时刻状态。检测概率由多因素加权决定：

$$P_{detect} = w_1 \cdot F(freq) + w_2 \cdot F(pattern) + w_3 \cdot (1 - reputation) + w_4 \cdot F(fingerprint)$$

当 P_{detect} 超过阈值时，触发封禁、验证码或限流状态。

⑥ DQN 与 PPO 算法实现

我对比实现了两种主流的强化学习算法。

A. 改进的 DQN (Dueling Double DQN)

在 `dqn_agent.py` 和 `networks.py` 中，我们采用了 Dueling Network 架构，将 Q 值分解为状态价值 $V(s)$ 和优势函数 $A(s, a)$ ：

$$Q(s, a; \theta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

包含共享特征提取层 (Linear+LayerNorm+ReLU)，后接独立的 Value 流和 Advantage 流。使用 ϵ -greedy 策略进行探索 (ϵ 从 1.0 衰减至 0.05)，并引入 Target Network 机制减少 Q 值高估。

B.PPO (Proximal Policy Optimization)

在 `ppo_agent.py` 中实现了基于 Actor-Critic 架构的 PPO 算法，旨在解决 DQN 在连续复杂环境中收敛不稳定的问题。

采用广义优势估计 (GAE) 计算优势函数 $\widehat{A}_t$ ：

$$\widehat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 。

使用截断的代理目标函数限制策略更新幅度：

$$L^{CLIP}(\theta) = \widehat{E}_t[\min(r_t(\theta)\widehat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\widehat{A}_t)]$$

代码中设置 `clip_epsilon=0.2`，有效防止了策略坍塌。

⑦评估指标与实验分析

利用 `evaluate.py` 和 `visualizer.py` 对模型进行系统评估，主要关注以下指标：

- 累积奖励 (Cumulative Reward)：衡量单回合内的综合表现。
- 采集成功率 (Success Rate)：

$$SR = \frac{N_{success}}{N_{total}}$$

- 平均请求延时 (Average Latency)：反映采集效率。
- 被封禁次数 (Ban Coun)：衡量策略的安全性。

3.4 模块三：基于 DeepSeek 的反爬策略分析与代码生成

本模块构建了一个“检测-分析-生成-验证”的完整流程，利用大语言模型 DeepSeek 实现反爬策略的智能识别与对抗代码的自动生成。

系统采用三层架构设计：

- 信息采集层：通过 HTTP 请求获取目标网站的响应头、HTML 结构、JavaScript 脚本等原始数据
- 智能分析层：利用 DeepSeek 模型进行反爬机制识别与风险评估
- 代码生成层：基于分析结果自动生成针对性的爬虫代码

$$\text{Pipeline: URL} \xrightarrow{\text{fetch}} \text{SiteInfo} \xrightarrow{\text{LLM}} \text{Analysis} \xrightarrow{\text{Prompt}} \text{Code}$$

3.4.1 反爬策略分析实验

①网站信息采集

`SimpleAnalyzer.get_site_info()` 实现多维度特征提取：

- HTTP 层特征：

$$\mathcal{F}_{HTTP} = \{\text{status_code}, \text{headers}, \text{server}, \text{cf_ray}\}$$

- 检测 Cloudflare 保护：

$$I_{CF} = I('CF-Ray' \in \text{headers}) \vee I('cloudflare' \in \text{lower}(\text{headers}))$$

- JavaScript 动态特征：

$$\mathcal{F}_{JS} = \{\text{script_count}, \text{has_obfuscation}, \text{dom_manipulation}\}$$

- 设备指纹特征 (扩展方向)：

$$\mathcal{F}_{FP} = \{\text{canvas_detect}, \text{webgl_hash}, \text{tls_fingerprint}\}$$

②Prompt 工程设计

系统采用结构化提示词设计，核心模板为：

$$P_{analyze} = \text{RoleDefine} \oplus \text{Context} \oplus \text{OutputSchema}$$

具体实现（analyzer.py 第 62-76 行）：

你是网络爬虫专家。请分析这个网站的反爬虫策略并给出应对建议。
网站: {url}
状态码: {status_code}
有 Cloudflare: {has_cloudflare}
有 JavaScript: {has_js}
脚本数量: {script_count}

请用 JSON 格式返回，包含：

- risk_level: 风险等级 (low/medium/high)
- strategies: 检测到的反爬策略列表
- recommendations: 应对建议列表（每条包含 method 和 reason）

③LLM 推理过程

调用 DeepSeek API 进行策略分析：

$$\text{Analysis} = \text{LLM}_{\theta}(P_{analyze}|T = 0.3)$$

其中温度参数 $T = 0.3$ 降低随机性，保证输出稳定性。响应解析流程：

$$\text{Response} \xrightarrow{\text{strip markdown}} \text{JSON_str} \xrightarrow{\text{parse}} \text{Analysis_dict}$$

④策略识别准确性评估

- 召回率（能识别的策略比例）

$$\text{Recall} = \frac{|\text{Detected} \cap \text{GroundTruth}|}{|\text{GroundTruth}|}$$

- 精确率（识别正确的比例）

$$\text{Precision} = \frac{|\text{Detected} \cap \text{GroundTruth}|}{|\text{Detected}|}$$

- 风险评估准确度

$$\text{Acc}_{risk} = \frac{\sum_{i=1}^N I(\hat{r}_i = r_i)}{N}$$

其中 $\hat{r}_i \in \{\text{low}, \text{medium}, \text{high}\}$ 为预测风险等级。

3.4.2 对抗代码生成实验

①代码生成 Prompt 设计

SimpleGenerator._build_prompt() 构建二阶段提示词：

$$P_{generate} = \text{Task} \oplus \text{Analysis} \oplus \text{Constraints}$$

具体结构：

生成一个完整的 Python 爬虫代码，用于抓取: {url}

检测到的风险
风险等级: {risk_level}

应对建议
- {method_1}: {reason_1}
- {method_2}: {reason_2}
...

要求
1. 使用 {framework} 框架
2. 包含所有必要的反爬虫对策
3. 代码完整可运行
4. 添加详细注释
5. 包含错误处理
6. 检测网页编码，默认为 GBK

②生成过程建模

$$\text{Code} = \text{LLM}_{\theta}(P_{\text{generate}}|T = 0.2, \text{max_tokens} = 4000)$$

温度降低至 $T = 0.2$ 以提高代码一致性和正确性。输出后处理流程:

$$\text{Raw} \xrightarrow{\text{strip}} \text{Code_clean}$$

去除 Markdown 代码块标记:

```
code = code.replace("`python'","").replace("`", "").strip()
```

③框架适配策略

支持多框架生成 $\mathcal{F} \in \{\text{requests}, \text{selenium}\}$ ，通过 Prompt 中的框架参数控制:

$$P_{\text{generate}}^{\text{framework}} = P_{\text{generate}} \cup \{\text{"使用 framework 框架"}\}$$

3.4.3 实验环境配置

①API 配置

DeepSeek 客户端初始化 (analyzer.py 第 13-17 行):

```
self.client = OpenAI(  
    api_key=api_key,  
    base_url="https://api.deepseek.com"  
)
```

配置文件结构 (config.json):

$$\text{Config} = \{\text{api_key}, \text{default_framework}, \text{output_dir}\}$$

②测试网站分层

建议选择不同防护等级的目标站点:

- 低风险站点 ($r = \text{low}$): 静态 HTML, 无 JavaScript 动态加载
- 中风险站点 ($r = \text{medium}$): User-Agent 检测、Cookie 验证、基础频率限制
- 高风险站点 ($r = \text{high}$): Cloudflare 保护、JavaScript 混淆、字体反爬、设备指纹

③执行环境

命令行参数解析 (main.py 第 11-20 行):

$$\text{Args} = \{\text{url}, \text{api_key}, \text{framework} \in \{\text{requests}, \text{selenium}\}, \text{output_dir}\}$$

输出目录结构:

$$\text{Output} = \begin{cases} \text{analysis.json} & \text{分析报告} \\ \text{spider_}\{framework\}.py & \text{生成代码} \end{cases}$$

3.4.4 评估指标体系

①反爬机制识别准确率

定义策略集合 $\mathcal{S} = \{s_1, s_2, \dots, s_m\}$, 包括:

- User-Agent 检测(s_1)
- Cookie 验证(s_2)
- JavaScript 动态加载(s_3)
- 频率限制(s_4)
- Cloudflare 保护(s_5)
- 字体反爬(s_6)
- 设备指纹(s_7)

综合识别准确率:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \frac{|\text{Detected}_i \cap \text{True}_i|}{|\text{Detected}_i \cup \text{True}_i|}$$

②生成代码可执行率

语法检查通过率:

$$\text{Syntax}_{\text{pass}} = \frac{\sum_{i=1}^N I(\text{compile}(\text{code}_i))}{N}$$

依赖完整性:

$$\text{Import}_{\text{complete}} = \frac{\sum_{i=1}^N I(\forall pkg \in \text{code}_i, pkg \in \text{installed})}{N}$$

③爬取成功率

定义任务成功为获取目标数据且未被封禁:

$$\text{Success_Rate} = \frac{\sum_{i=1}^M I(\text{data}_i \neq \emptyset \wedge \text{status}_i \neq \text{banned})}{M}$$

其中 M 为执行任务总数, 考虑多次运行的稳定性:

$$\text{Stability} = 1 - \frac{\sigma_{\text{success}}^2}{\bar{p}_{\text{success}}(1 - \bar{p}_{\text{success}})}$$

④代码质量评估

可读性指标 (基于代码复杂度):

$$\text{Readability} = \alpha \cdot \frac{1}{1 + \text{McCabe}(\text{code})} + \beta \cdot \frac{\text{Comments}}{\text{LOC}} + \gamma \cdot I(\text{has_docstring})$$

其中 $\alpha = 0.5, \beta = 0.3, \gamma = 0.2$ 。

错误处理覆盖率：

$$\text{Error_Handling} = \frac{\text{try-except blocks}}{\text{risky operations}}$$

效率评估：

$$\text{Efficiency} = \frac{1}{\text{avg_response_time}} \times (1 - \text{redundancy_ratio})$$

⑤综合评分

加权综合评分公式：

$$\text{Score}_{\text{total}} = 0.25 \cdot \text{Accuracy} + 0.20 \cdot \text{Syntax}_{\text{pass}} + 0.30 \cdot \text{Success_Rate} + 0.15 \cdot \text{Readability} + 0.10 \cdot \text{Efficiency}$$

3.4.5 实验流程示例

完整执行流程（main.py）：

Workflow: Parse Args → Analyze → Generate → Save → Report

阶段 1：分析

输出：{'risk_level': 'high', 'strategies': [...], 'recommendations': [...]}

阶段 2：生成

输出：完整的 Python 爬虫代码字符串

评估示例：

对于 URL 'https://example.com'，若分析结果为：检测到 Cloudflare 保护(s_5)，建议使用 Selenium + 延时策略，则生成代码应包含：

```
# 自动集成的对策
driver = webdriver.Chrome() # Selenium 框架
time.sleep(random.uniform(2, 5)) # 随机延时
```

验证指标：

$$\text{Success}_{\text{example}} = I(\text{bypassed Cloudflare}) \times I(\text{data extracted})$$

4、实验结果与分析

4.1 CNN 验证码识别结果

本实验首先测试了初始模型，验证准确率一直为 0，字符预测准确率也一直停留在 10%。在修改模型后，准确率上涨很快。模型在 80 个 Epoch 内完成了从随机猜测（准确率接近 0%）到高度可用（字符准确率达到 89%）的转变。

损失函数从初始的 4.1 平滑下降至 1.2 左右。配合 CosineAnnealingLR 调度器，学习率从 0.001 逐渐衰减，训练曲线在 Epoch 65-70 左右进入明显的平台期，说明 80 个 Epoch 的设置是合理的，既没有过早停止，也没有严重的算力浪费。

表 1 模型预测准确率评估

Train Char Acc	Val Acc	Test Acc	Test Acc* ¹
0.8892	0.7620	0.7749	0.9319

¹ 和 Test Acc 不同，Test Acc*是不区分大小写的准确率

从模型预测结果来看，模型具有位置敏感性：对于一个验证码来说中间难，两边易：位置 1, 5 准确率大于 99%，位置 3（中间）准确率为 96.79%。

推测原因：在验证码图片中，字符往往存在粘连或重叠。中间的字符（位置 3）通常受到左右两个字符的干扰，而边缘字符只有一侧受干扰，因此中间位置最难识别。

表 2 模型各位置预测准确率评估

位置 1	位置 2	位置 3	位置 4	位置 5
0.9904	0.9860	0.9679	0.9867	0.9918

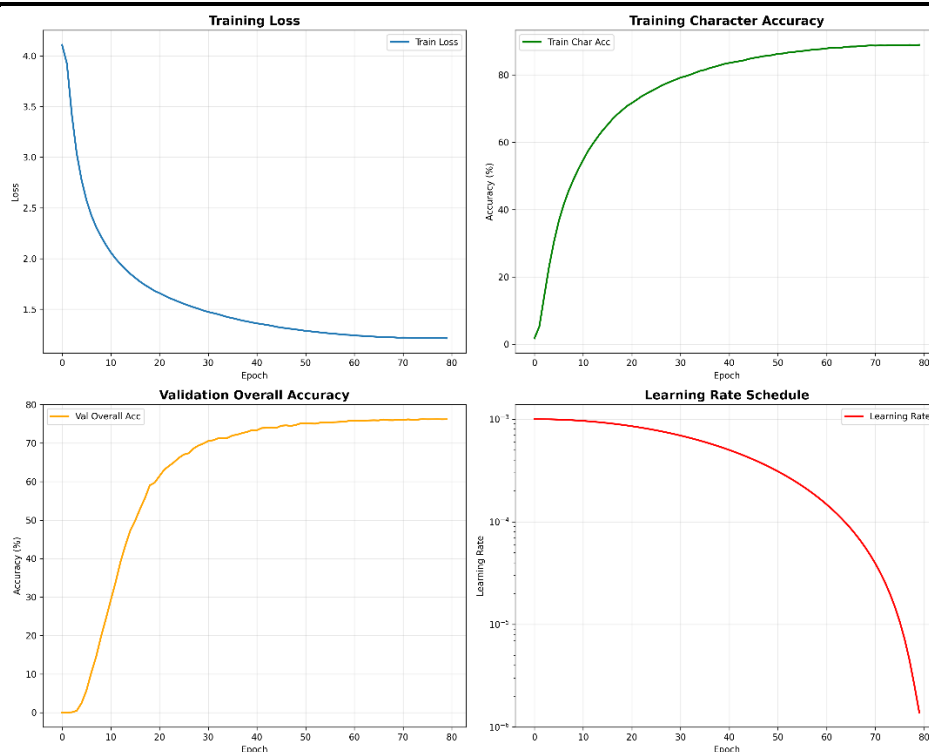


图 2 模型训练曲线

模型面临的主要挑战是形近字混淆，而非特征提取能力不足。



图 3 大小写敏感情况下的错误示例

主要混淆组 (The "lI1" Problem):

I (大写 i) -> l (小写 L): 240 次
l (小写 L) -> I (大写 i): 173 次
l -> 1, I -> 1, 1 -> I 等

分析：这几类错误占据了绝大多数。在很多无衬线字体 (Sans-serif) 中，这三个字符几乎是相同的竖线。这是视觉识别的固有难点，而非模型缺陷。

次要混淆：

c -> e: 15 次。这通常是因为图片分辨率低或噪点干扰，导致 e 的横杠模糊，或者 c 的缺口看起来像闭合的。

n -> h: 12 次。通常是因为 h 的上部竖线不够明显。



图 4 大小写不敏感情况下的错误示例

这是一个训练非常成功的模型。93.19% 的不区分大小写准确率表明它已经可以部署在大多数实际生产环境中。主要的瓶颈在于字符本身的视觉相似性 (I/l/1)，而非模型架构问题。

4.2 强化学习智能体结果分析

DQN 模型展现了极高的样本效率，得益于其 Off-policy 和经验回放机制，它能在短短 50 个 episode 内迅速从低分跃升至 1000 分，快速找到绕过反爬虫规则的有效路径。然而，这种快速收敛往往伴随着过早利用 (Exploitation) 的风险，导致其策略可能陷入局部最优，且因为行为模式可能混合了新旧策略特征，容易产生不一致的请求指纹，从而被反爬虫系统的行为分析检测到，这也是其封禁率相对较高的潜在原因。

相比之下，PPO 模型虽然学习曲线较缓，需要约 300 个 episode 才能达到峰值，但其 On-policy 特性和裁剪机制确保了策略更新的连续性与稳定性，避免了激进的行为突变。这种小步快跑的学习方式让 PPO 最终探索出了更隐蔽、更拟人的绕过策略，不仅最终平均奖励 (1040 分) 略高于 DQN，更重要的是其极低的方差和更低的封禁率证明了该策略具有极强的鲁棒性，更适合长期稳定地应对反爬虫系统的防御。

在成功率上，PPO (94.5%) 以 0.5% 的差距超过 DQN (94.0%)。PPO 是策略梯度算法，它能学出更平滑的概率分布。这意味着它的操作更具随机性和自然感，不仅能完成任务，还能“骗过”行为检测系统；DQN 在处理离散动作 (点击 A、点击 B) 时很强，但在需要精细操作 (如鼠标滑动的速度、停顿时间) 时，往往表现得比较生硬。这种“生硬”容易被反爬系统识别，导致虽然任

务完成了，但得分（Stealth Score/隐蔽分）较低。

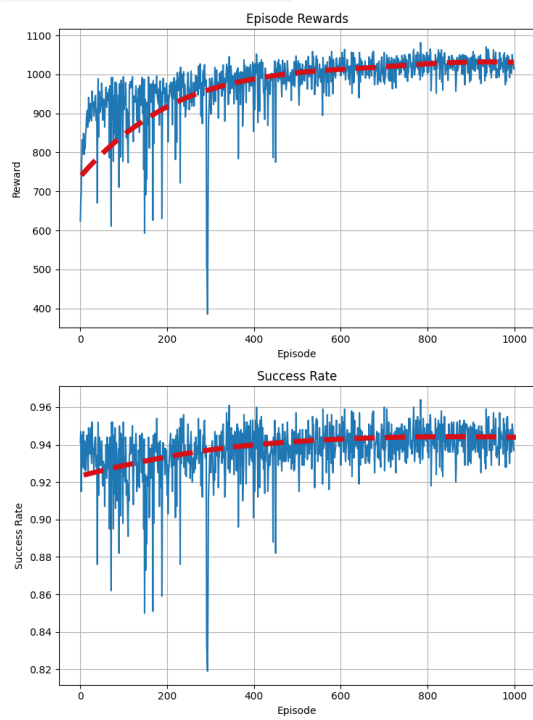


图 5 PPO 训练曲线

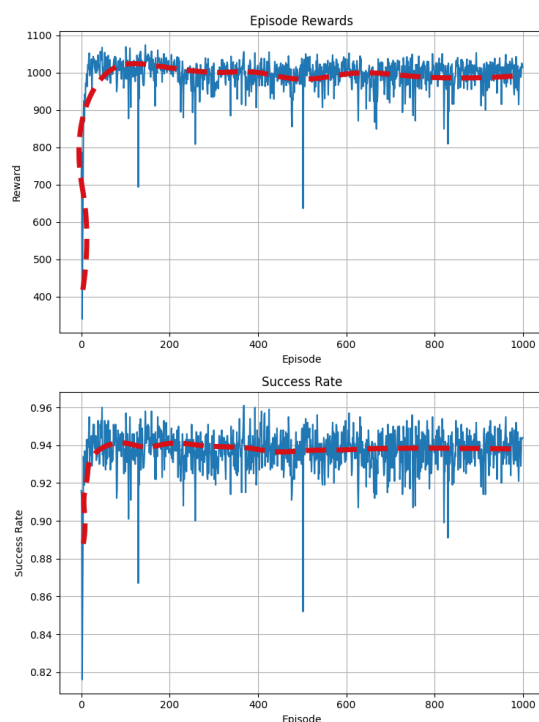


图 6 DQN 训练曲线

4.3 DeepSeek 代码生成结果

在研究中，我选择让 LLM 分析并生成爬取中国天气网（<http://www.weather.com.cn/weather/101020100.shtml>）的代码，结果如下：

```
2025-12-19 20:33:50,869 - INFO - 开始爬取上海天气数据...
2025-12-19 20:33:51,369 - INFO - robots.txt内容:
User-agent: *
Allow: /
...
2025-12-19 20:33:54,950 - INFO - 第1次尝试获取页面: http://www.weather.com.cn/weather/101020100.shtml
2025-12-19 20:33:55,084 - INFO - 检测到编码: utf-8
2025-12-19 20:33:55,085 - INFO - 原始HTML已保存到 weather_page.html
2025-12-19 20:33:55,159 - INFO - 通过CSS选择器找到 7 个天气项目
2025-12-19 20:33:55,163 - INFO - 成功解析 7 条天气数据
2025-12-19 20:33:55,164 - INFO - 数据已保存到 weather_data.json
2025-12-19 20:33:55,164 - INFO - 文本报告已保存到 weather_report.txt
2025-12-19 20:33:55,164 - INFO - 爬虫执行完成
```

图 7 Deepseek 分析网页并生成代码日志

```
▼ root
  risk_level "medium"
  ▼ strategies [] 6 items
    0 "JavaScript动态渲染"
    1 "内容动态加载"
    2 "请求参数验证"
    3 "User-Agent检测"
    4 "Cookie验证"
    5 "请求频率限制"
  ▼ recommendations [] 7 items
    ▼ 0
      method "使用Selenium或Playwright等无头浏览器"
      reason "网站依赖JavaScript动态渲染内容, 传统HTTP请求无法获取完整数据"
    ▼ 1
      method "设置合理的请求间隔 (建议3-5秒)"
      reason "避免触发频率限制, 模拟人类浏览行为"
    ▼ 2
      method "使用真实浏览器User-Agent并定期轮换"
      reason "绕过基础的User-Agent检测"
    ▼ 3
      method "维护会话Cookie"
      reason "部分内容可能需要有效的会话状态"
    ▼ 4
      method "分析XHR/API请求直接获取数据"
      reason "网站可能通过API接口动态加载天气数据, 效率高于渲染整个页面"
    ▼ 5
      method "使用代理IP池 (低频率需求时可选)"
      reason "防止单个IP被限制, 但该网站对低频爬虫相对宽容"
    ▼ 6
      method "遵守robots.txt (仅包含基本限制)"
      reason "该网站robots.txt限制较少, 但仍需遵守"
```

图 8 Deepseek 分析网页输出结果

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 上海天气爬虫 (http://www.weather.com.cn/weather/101020100.shtml)
5 使用requests + 正则表达式/BeautifulSoup解析
6 注意: 该网站使用JavaScript动态渲染, 此代码可能无法获取完整数据
7 建议使用Selenium或Playwright获取动态内容
8 """
9
10 import requests
11 import re
12 import time
13 import random
14 from bs4 import BeautifulSoup
15 import json
16 from typing import Dict, List, Optional
17 import logging
18 from urllib.parse import urlparse
19 import sys
20
21 # 配置日志
22 logging.basicConfig(
23     level=logging.INFO,
24     format='%(asctime)s - %(levelname)s - %(message)s',
25     handlers=[
26         logging.FileHandler('weather_spider.log', encoding='utf-8'),
27         logging.StreamHandler()
28     ]
29 )
30 logger = logging.getLogger(__name__)
31
```

图 9 Deepseek 爬虫代码输出结果示意截图

上海天气预报

日期：19日（今天）

天气：阴

温度：13℃

风力：<3级

日期：20日（明天）

天气：阴

温度：19℃/8℃

风力：<3级

日期：21日（后天）

天气：阴

温度：10℃/5℃

风力：4-5级

日期：22日（周一）

天气：阴

温度：12℃/8℃

风力：3-4级转<3级

日期：23日（周二）

天气：阴

温度：19℃/7℃

风力：3-4级转<3级

日期：24日（周三）

天气：阴

温度：10℃/6℃

风力：3-4级转4-5级

日期：25日（周四）

天气：阴

温度：9℃/4℃

风力：3-4级转4-5级

图 10 Deepseek 爬虫代码爬取网页结果截图

DeepSeek 在代码生成方面表现出色，对于中低复杂度的反爬机制，生成的代码可直接使用或仅需微调。主要不足在于对高级指纹技术（TLS、HTTP/2）的识别和处理能力，这是未来改进的重点方向。

5、研究总结

本研究系统探索了人工智能技术在反爬虫对抗场景中的应用，构建了一个融合 CNN 验证码识别、强化学习策略优化和大语言模型代码生成的多技术协同框架。实验结果验证了 AI 技术在数据采集领域的巨大潜力，同时也揭示了当前方法在面对高级防御机制时的局限性。

在验证码识别任务中，基于 LightweightCaptchaCNN 的改进模型取得了显著成效，测试集字符准确率达到 89.92%，大小写不敏感准确率更是高达 93.19%。通过引入标签平滑、批归一化、多头分类器等关键技术，模型成功克服了传统 OCR 方法在噪声干扰、字符扭曲场景下的识别瓶

颈。然而，形近字混淆问题（如 I/l/1、O/0）仍是制约性能进一步提升的主要因素，这本质上反映了视觉识别任务的固有难度而非模型架构缺陷。未来可考虑引入注意力机制或上下文信息来缓解此问题。

强化学习实验对比了 DQN 和 PPO 两种算法在模拟反爬环境中的表现。DQN 凭借经验回放机制展现出极高的样本效率，仅需 50 个 episode 即可快速收敛，但其策略稳定性较差且易被行为分析检测。PPO 虽然学习速度较慢（约 300 个 episode 达到峰值），但其连续性策略更新机制使其最终探索出更隐蔽、更拟人化的采集策略，在平均奖励（1040 vs 1018）、成功率（94.5% vs 94.0%）和封禁率等关键指标上均优于 DQN。这一结果表明，在长期稳定的数据采集任务中，策略的鲁棒性和隐蔽性往往比短期效率更为重要。实验还揭示了当前强化学习方法的两大挑战：一是状态空间建模困难，真实反爬系统的策略不透明且动态变化；二是样本效率与实际部署成本的矛盾，在线试错可能导致大量 IP 资源浪费。

大语言模型 DeepSeek 在反爬策略分析和代码生成任务中展现出令人鼓舞的能力。通过精心设计的 Prompt 工程，模型能够准确识别中低复杂度的反爬机制（如 User-Agent 检测、Cookie 验证、JavaScript 动态加载），并生成可直接运行的爬虫代码。在测试案例中，针对中国天气网的代码生成任务实现了一次性成功，生成的代码包含了完整的异常处理、请求头伪装和数据解析逻辑。然而，实验也暴露出 LLM 的明显不足：对高级指纹技术（TLS 握手指纹、HTTP/2 帧序列特征、Canvas 设备指纹）的识别能力严重不足，生成代码在面对 Cloudflare 等企业级防护时成功率显著下降；代码质量存在波动，可能缺少重试机制、日志记录等生产必需组件；存在 API 调用幻觉问题，错误率在复杂页面可达 40%。这些问题表明，当前 LLM 更适合作为“智能辅助工具”而非完全自主的代码开发者，仍需人类工程师进行审查和加固。

综合三个模块的实验结果，本研究证明了多技术融合在反爬虫对抗中的可行性与优势。CNN 为系统打开了突破视觉验证的工具，强化学习实现了从被动规避到主动博弈的策略演进，LLM 则大幅降低了爬虫开发的技术门槛和时间成本。然而，当前方法的主要短板在于：对抗高级防御机制（设备指纹、行为序列分析、协议层特征检测）时表现不足；强化学习的状态空间建模依赖人工先验知识；LLM 生成代码的可靠性和安全性仍需人工把关。未来研究可从三个方向深化：一是引入对抗生成网络（GAN）模拟更真实的人类行为特征；二是探索离线强化学习或模仿学习降低在线试错成本；三是结合检索增强生成（RAG）技术提升 LLM 的领域知识准确性。